

Agentic Environment

Security Review Framework

A comprehensive guide to security considerations beyond prompt injection for teams building and reviewing AI agent systems.

Contents

1.	Agent Identity & Authorisation	3
2.	Tool & Action Security	4
3.	Data Handling & Leakage	5
4.	Memory & State Security	6
5.	Multi-Agent Trust & Orchestration	7
6.	Observability & Audit	8
7.	Supply Chain for AI Components	9
8.	Human-Agent Interaction Security	10
9.	Security Review Questions	11

Introduction

Prompt injection receives most of the attention in agentic AI security — but it is only one piece of a much larger threat surface. Agentic systems introduce security problems that simply do not exist in traditional software: agents that act autonomously, hold persistent memory, call external tools, spawn sub-agents, and operate on behalf of users across long multi-step tasks.

This document maps eight threat domains that every security reviewer should evaluate when assessing an agentic system. Each domain includes the key considerations, their risk severity, and the rationale behind each control. A set of review questions is provided at the end to structure a structured security assessment.

Risk levels		
Critical	High	Medium
Immediate risk of severe harm. Must be addressed before launch.	Significant risk. Address in current sprint or milestone.	Meaningful risk. Address in near-term roadmap.

Agent Identity & Authorisation

Agents often inherit the full permission set of the user they act on behalf of, or share a single service account across many agent instances. This creates an IAM model that is far too permissive for autonomous systems. Each agent — ideally each agent session — must be treated as its own principal with scoped, short-lived credentials.

Agent over-privilege **Critical**

Agents often inherit the full permission set of the user they act on behalf of. A reading agent shouldn't have delete rights. An email summariser shouldn't have send rights. Audit tool scope against stated purpose — ambient authority is the most common IAM failure in agentic systems.

Shared agent identities **Critical**

Multiple agents sharing one service account means a compromise of any single agent compromises the credential for all. Each agent — ideally each agent session — needs its own principal with scoped, short-lived credentials.

Agent-to-agent trust escalation **High**

In multi-agent systems, Agent A may invoke Agent B. If Agent B trusts Agent A blindly, an attacker who compromises A can use it to direct B to take privileged actions. Agent-to-agent calls need authentication and authorisation, not implicit trust.

Credential persistence **High**

Long-lived API keys or tokens stored in agent memory or environment variables are a persistent risk. Agents should use short-lived tokens (OIDC, STS) that expire after the task, not long-lived credentials that persist across sessions.

Tool & Action Security

Prompt injection gets the headline, but the real harm comes from what the agent can do with a successful injection. The threat model must start with: 'what's the worst action this agent can take?' and work backwards. Irreversibility is a first-class security property.

Irreversible actions without confirmation **Critical**

Agents that can send emails, delete files, execute code, or make API calls without human confirmation create catastrophic failure modes. A single injection or hallucination can cause permanent damage. If the action can't be undone, the design must require explicit human approval.

Tool poisoning **Critical**

In systems where tools are discovered dynamically (MCP, tool registries), a malicious tool definition can override or shadow a legitimate one. The agent calls what it thinks is a safe tool but executes attacker code. Tool sources must be allowlisted and their definitions verified.

Code execution environments **Critical**

Agents that can write and execute code are a high-severity attack surface. Sandbox escapes, resource exhaustion, and data exfiltration via code execution are all realistic. Code execution tools need strict isolation: no network access by default, no file system access outside a scratch space, hard resource limits.

SSRF via agent-controlled URLs **High**

If an agent can make HTTP requests to attacker-controlled URLs, it can be used to probe internal infrastructure, reach cloud metadata endpoints (169.254.169.254), or exfiltrate data. All agent-initiated outbound requests need URL allowlisting or at minimum blocklisting of internal ranges.

Data Handling & Leakage

Everything in an agent's context window — user messages, retrieved documents, tool outputs — may be logged, used for fine-tuning, or visible to model providers. Data minimisation in the context is both a security and compliance requirement.

PII in context windows **Critical**

Everything in an agent's context window may be logged and visible to model providers. Sensitive data (PII, credentials, health data) entering the context window must be explicitly considered and minimised.

RAG data access control **Critical**

Retrieval-augmented generation systems retrieve documents based on semantic similarity, not ACLs. A user asking a question may receive retrieved content from documents they shouldn't have access to. The retrieval layer must enforce the calling user's permissions — not just return the most semantically relevant result.

Cross-tenant data leakage **Critical**

In multi-tenant agentic products, one customer's agent must not retrieve, infer, or act on another customer's data. Shared vector stores, shared LLM context, and shared tool instances are all potential leakage surfaces.

Model output as exfiltration channel **High**

An injected agent can exfiltrate data by encoding it in its output — base64, URL parameters, or markdown image tags that make outbound requests. Output must be scanned for encoded data and suspicious outbound references before delivery.

Embedding inversion **Medium**

Vector embeddings can sometimes be partially inverted to recover original text. Sensitive documents stored as embeddings are not fully anonymised — they may leak semantic content or specific text. This is an emerging risk area.

Memory & State Security

Agents with persistent memory introduce a class of attack — memory poisoning — that has no equivalent in traditional software. An attacker can cause the agent to store a malicious 'memory' that persists across sessions and influences all future behaviour.

Memory poisoning **Critical**

An attacker embeds a malicious instruction in content the agent stores as a memory. Future sessions retrieve the poisoned memory and act on it — a sleeper attack that persists across conversations. Memory contents must be treated as untrusted data, not trusted instructions.

Memory store access control **High**

If multiple users share a memory backend, or if memory is scoped to user ID without proper isolation, one user's agent can read or overwrite another's memories. Memory stores need the same ACL rigour as any database.

Context window manipulation **High**

In long-running agentic sessions, early content influences later decisions. An attacker who can influence content early in a session can bias the agent's behaviour for the entire session — even after their specific message is no longer in the current context window.

State persistence after task completion **Medium**

Agent state, credentials, and in-context data should be cleared after task completion. Persistent state across sessions increases the risk that sensitive data from one session influences or leaks into another.

Multi-Agent Trust & Orchestration

As soon as you have agents calling other agents, you have a principal hierarchy problem. Agent B receives a request from Agent A. If Agent B trusts Agent A because 'it's one of ours,' you've created a confused deputy vulnerability. Every agent action must be traceable back to explicit human authorisation.

Orchestrator compromise **Critical**

In multi-agent systems, a compromised orchestrator can direct sub-agents to take harmful actions. The orchestrator is a high-value target — it must be treated with the same security rigour as a privileged service, not as a trusted coordinator.

Confused deputy attacks **High**

Agent A requests Agent B to perform an action on its behalf. Agent B, trusting the peer agent, complies without verifying that the original human user actually authorised the action. Each agent must verify authorisation against the original human principal, not the requesting agent.

Agent loop / infinite recursion **High**

Agents calling agents can create loops — intentionally through injection, or accidentally. Infinite loops consume resources, cause denial of service, and may incur large API costs. Depth limits, cycle detection, and budget caps are required.

Inter-agent communication integrity **High**

Messages between agents in a distributed system can be intercepted and modified. Agent-to-agent communication needs authentication (each agent can prove its identity) and integrity protection (messages can't be tampered in transit).

Observability & Audit

Unlike deterministic code, the same input can produce different agent behaviour across runs. Security controls must be evaluated against the range of possible agent behaviours. And without a complete audit trail, forensics after an incident is impossible.

Missing action audit trail **Critical**

Every tool call, external request, file access, and decision must be logged immutably with full context: session ID, user ID, input that triggered the action, tool invoked, parameters, response, timestamp. Without this, forensics after an incident is impossible.

Non-determinism without explanation **High**

Unlike deterministic code, the same input can produce different agent behaviour across runs. Controls need to be evaluated against the range of possible agent behaviours, not just one observed run. Testing must be probabilistic, not point-in-time.

Prompt log retention **High**

Full prompts (system prompt + context + user message) sent to the LLM must be logged for security forensics and compliance. Many teams log only outputs. Without the full prompt, you cannot reconstruct what instruction led to a harmful action.

Cost and resource abuse detection **Medium**

Agents driven by injection can be made to make excessive API calls, consume large compute budgets, or trigger expensive operations. Anomalous cost spikes are a detection signal — monitor and alert on them alongside security events.

Supply Chain for AI Components

The AI supply chain extends traditional software supply chain risks into new territory. Model weights can be backdoored. Prompt templates are code. Third-party tool plugins need the same review rigour as any third-party library — plus additional scrutiny for what data they receive and what actions they can take.

Model weight integrity Critical

If you load a model from a registry (Hugging Face, etc.), the weights could be tampered — containing backdoors that activate on specific trigger inputs. Model provenance must be verified: checksums, signatures, and trusted source attestation before deploying any model.

Prompt template injection High

System prompts and prompt templates are code — they're part of the trusted instruction layer. If stored in a database or CMS and modifiable by a non-security-reviewed process, an attacker gaining write access to the template store can compromise every agent that uses it.

Third-party tool / plugin risk High

MCP servers, LangChain tools, and plugin ecosystems extend agent capabilities via third-party code. These need the same security review process as any third-party library — plus additional scrutiny for what data they receive and what actions they can take.

Fine-tuning data poisoning High

If the model is fine-tuned on customer data or interaction logs, poisoned training data can embed backdoors — the model behaves normally in evaluation but triggers on specific inputs in production. Training data provenance and cleaning is a security concern, not just a data quality one.

Human-Agent Interaction Security

As users become accustomed to agents working correctly, they start rubber-stamping confirmation prompts without reading them. Security design must account for automation bias: high-stakes confirmations must require genuine attention, not just a click-through.

Intent verification **High**

The agent's understanding of what the user wants can drift from what the user actually intended, especially in long multi-step tasks. Before irreversible actions, the agent should re-confirm its understanding of intent — not execute based on its interpretation from several turns ago.

Authorisation laundering **High**

'The user has access to this file, so I can send it' is not the same as 'the user approved sending this file.' Agent actions need active authorisation for the specific action, not just ambient access rights.

Deceptive agent outputs **High**

A compromised or injected agent might produce outputs designed to manipulate the human supervisor — claiming a dangerous action is safe, hiding what it actually did, or presenting false summaries. Human review of agent outputs should include verification, not just reading.

Automation bias **Medium**

As users trust agents more, they rubber-stamp confirmations without reading them. High-stakes confirmations should be structured to require genuine attention — specific detail the user must acknowledge, not a generic click-through.

Security Review Questions

Use these questions to structure a security review of any agentic system. They map directly to the eight domains covered in this document.

Agent identity

Does each agent have its own identity and credential, or do agents share service accounts?

Why it matters: Shared credentials mean one compromised agent exposes all others.

Agent identity

What happens to agent credentials when the task completes — are they revoked automatically?

Why it matters: Credentials persisting after task completion are a standing attack surface.

Tool security

Which agent actions are irreversible, and do all of them require explicit human confirmation?

Why it matters: A single injection or hallucination causing an irreversible action can be catastrophic.

Tool security

Where do tool definitions come from — hardcoded or dynamically discovered? If dynamic, how are they verified?

Why it matters: Dynamic tool discovery is a supply chain risk. A malicious definition can shadow a legitimate tool.

Tool security

If the agent can execute code, is the execution environment sandboxed with no network, no file system, and hard resource limits?

Why it matters: Code execution without sandboxing is a full system compromise waiting to happen.

Tool security

Can the agent make outbound HTTP requests? Is there a URL allowlist or block for internal network ranges?

Why it matters: Unrestricted outbound requests enable SSRF attacks targeting internal infrastructure.

Data handling

What data enters the agent context window — is any of it PII, credentials, or regulated data?

Why it matters: Everything in the context window may be logged and visible to model providers.

Data handling

Does the RAG retrieval layer enforce the calling user access permissions before returning documents?

Why it matters: Semantic search does not respect ACLs. A user can receive documents they should not access.

Data handling

In a multi-tenant system, how is customer data isolated at the vector store level, not just at the application level?

Why it matters: Application-level isolation can be bypassed. Isolation must be enforced at the data store level.

Memory

Does the agent have persistent memory? If so, are memory contents treated as untrusted data or trusted instructions?

Why it matters: Memory poisoning is a sleeper attack — a malicious instruction stored in memory executes in future sessions.

Memory

Is there access control on who can read and write the agent memory store?

Why it matters: A shared memory backend without ACLs allows one user agent to poison another memories.

Multi-agent

When Agent A calls Agent B, does Agent B verify the original human authorisation or trust the calling agent?

Why it matters: Confused deputy attacks exploit this gap — a compromised orchestrator can direct sub-agents without human approval.

Multi-agent

Is there a depth limit on agent-to-agent invocations? What prevents infinite loops?

Why it matters: Agent loops can cause denial of service, cost explosion, and are a viable injection attack vector.

Observability

Is every tool call logged with full context — what triggered it, parameters used, response received?

Why it matters: Without a complete audit trail, forensics after an incident is impossible.

Observability

Are full prompts — not just outputs — retained for security forensics?

Why it matters: You cannot reconstruct why an agent took a harmful action without the full prompt that preceded it.

Supply chain

If using a third-party or open-source model, have the weights been verified against a known-good checksum?

Why it matters: Tampered model weights can contain backdoors invisible in normal testing.

Supply chain

Are system prompts and prompt templates versioned, reviewed, and access-controlled like code?

Why it matters: Write access to a prompt template is equivalent to write access to the agent behaviour.

Human-agent

For high-stakes actions, does the agent re-confirm user intent rather than acting on an old interpretation?

Why it matters: Intent drift across a long multi-step task is real — agent understanding can diverge from user intent.

Human-agent

Are confirmation prompts for dangerous actions specific enough to require genuine attention, or generic click-throughs?

Why it matters: Automation bias means users rubber-stamp generic confirmations. High-stakes gates must require acknowledgement of specific details.